Introduction to Applied Machine Learning

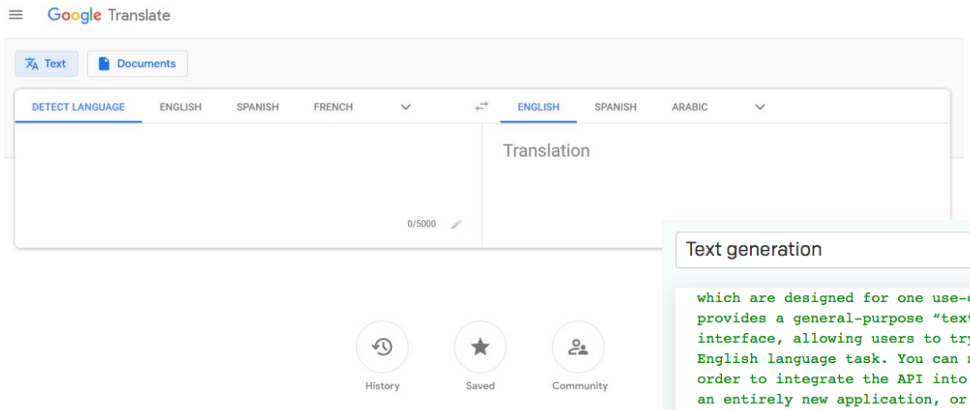
08: Sequence Models and the Transformer Architecture

🎓 Machine Learning Adventure 🚀

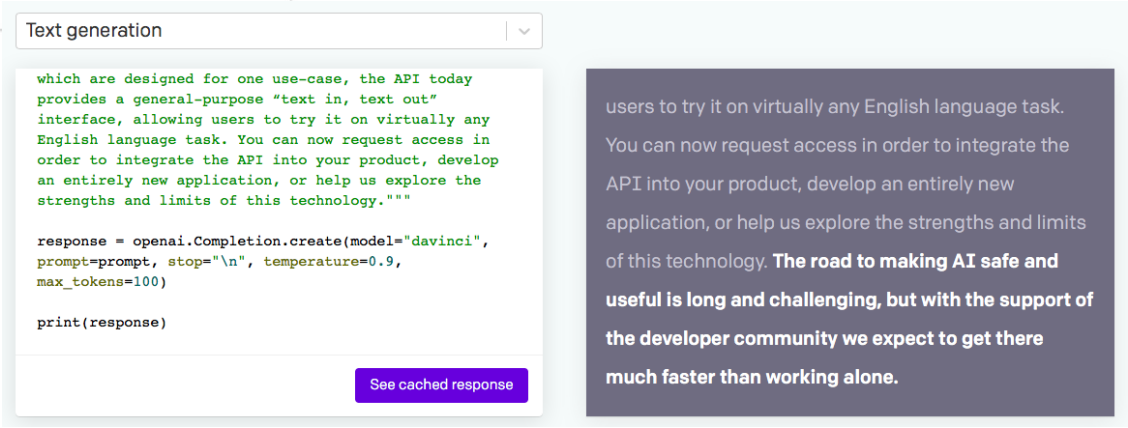
1. 🤖 Introduction to Machine Learning
2. 🧠 Artificial Neural Networks and Computer Vision
3. 🔤 Transformers and Language Models
4. 🎨 GANs and Diffusion for Image Generation
5. 🎮 Reinforcement Learning and Behavior
6. 🏆 Final Projects

Applications in Natural Language Processing (NLP)

Machine Translation



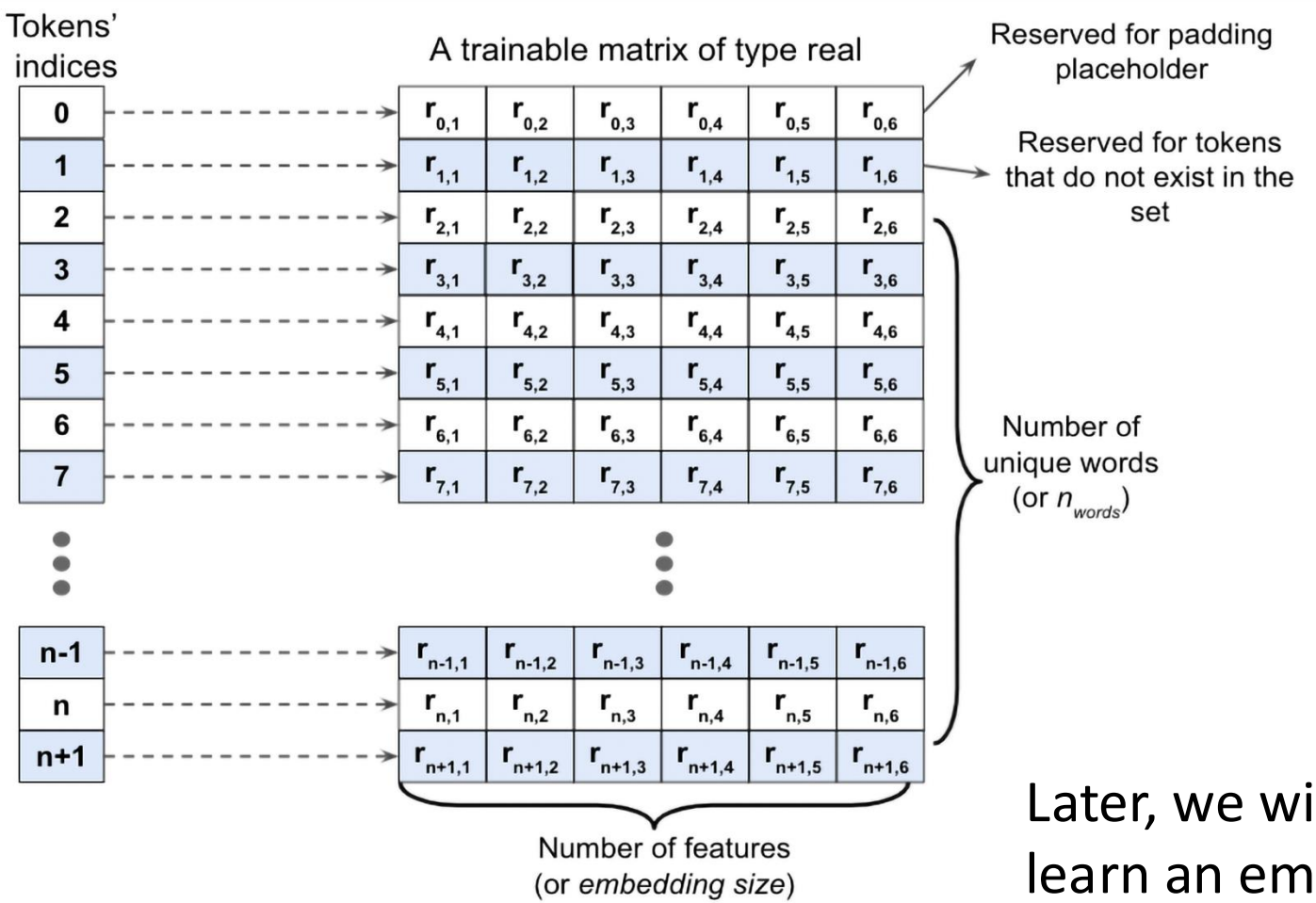
Text Generation



Speech Recognition

Preface: Word Embeddings

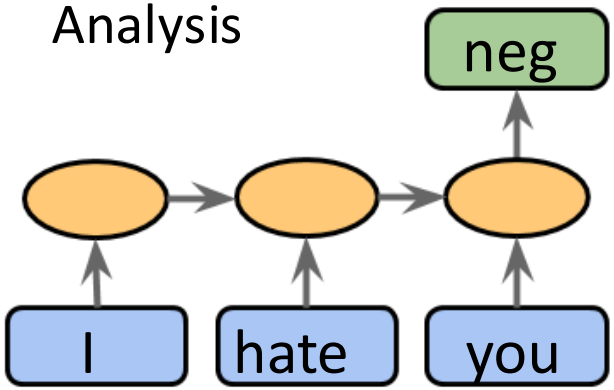
Given a vocabulary, generally represent a “word” (or token) as a vector (one hot encoding → learned embedding).



Later, we will see how to learn an embedding.

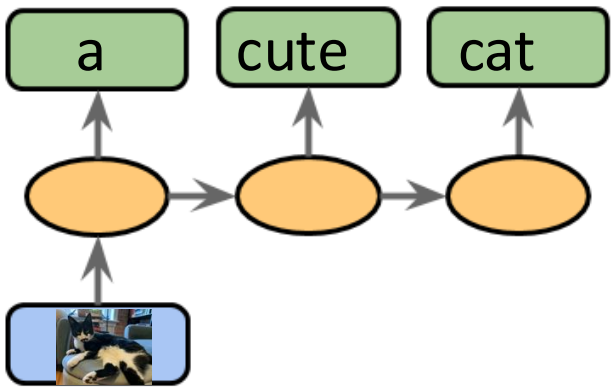
Example Sequence Modeling Tasks

Example: Sentiment Analysis



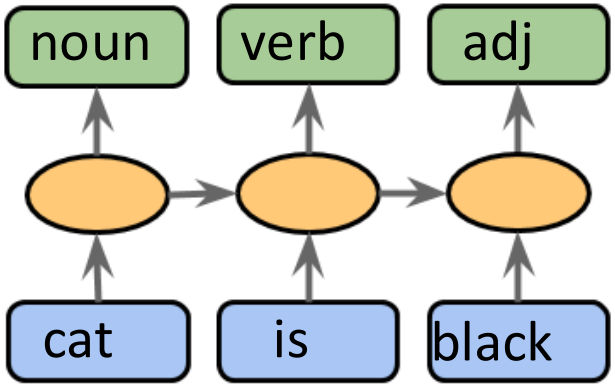
Many-to-one

Example: Image Captioning



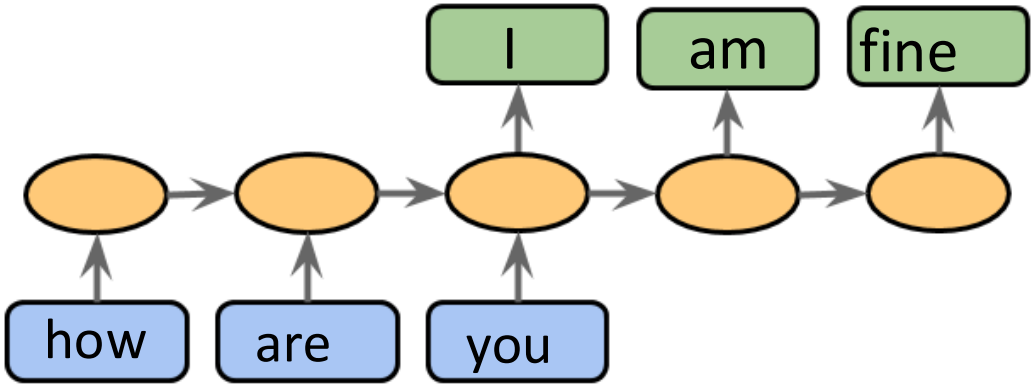
One-to-many

Example: Part of Speech Tagging



Many-to-many

Example: Question Answering



Many-to-many

Language Models and Conditional Probability

A statistical model of language can be represented by the conditional probability of the next token, given all the previous ones, since

$$\Pr(\boldsymbol{x}_1, \dots, \boldsymbol{x}_N) = \prod_{i=1}^{N} \Pr(\boldsymbol{x}_i | \boldsymbol{x}_1, \dots, \boldsymbol{x}_{i-1})$$

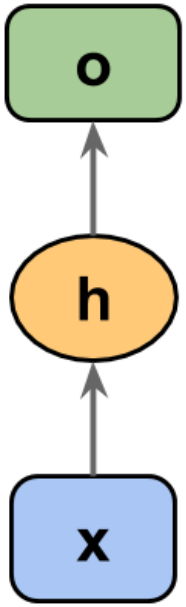
A model that predicts $\Pr(\boldsymbol{x}_i | \boldsymbol{x}_1, \dots, \boldsymbol{x}_{i-1})$ is called a **language model** (specifically a **causal** language model).

Example: GPT series models.

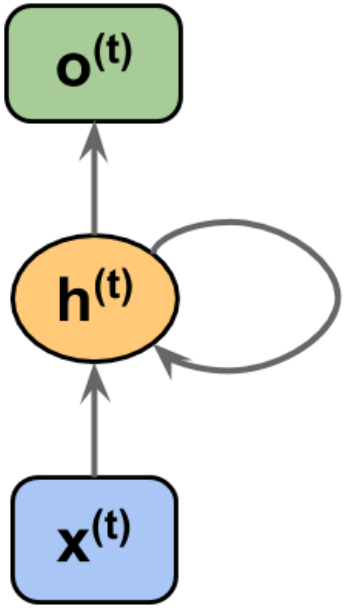
Feedforward vs Recurrent Neural Networks

State of the art sequence model architectures through 2016 were **recurrent** (RNNs and the related LSTM).

A standard feedforward network



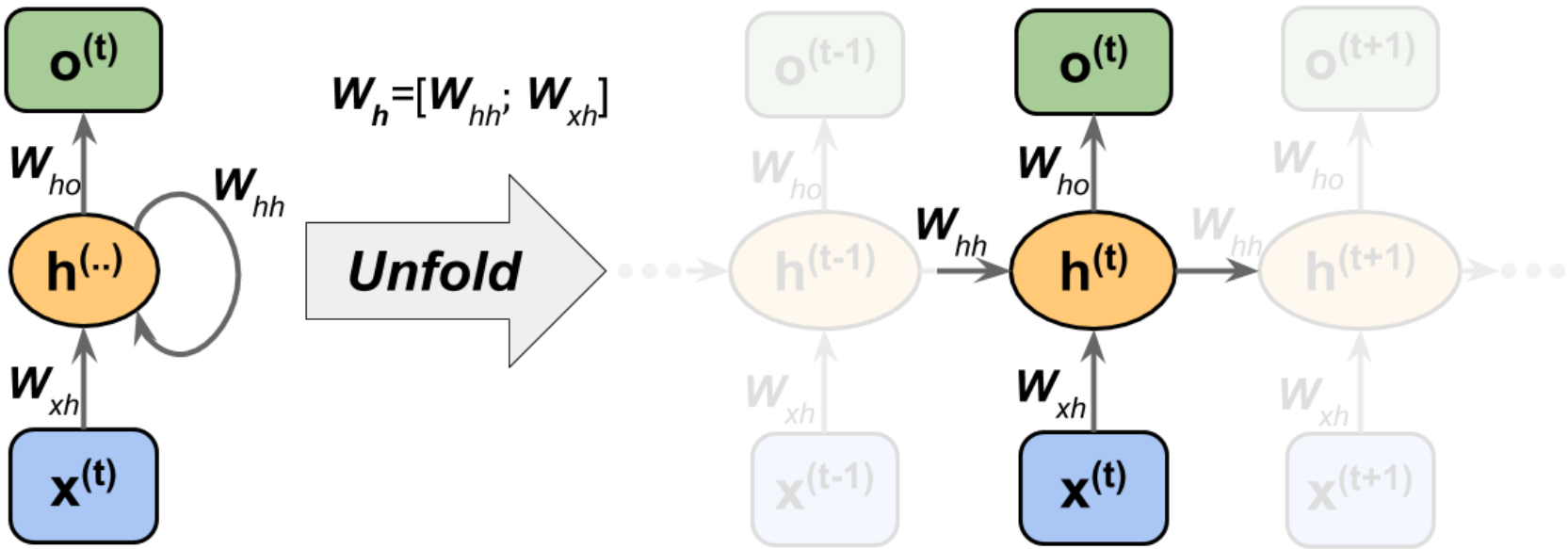
Recurrent neural network



In which the hidden units are a function of the input.

In which the hidden units are a function of the input **and** the previous values of the hidden units.

Unfolding a Recurrent Neural Network

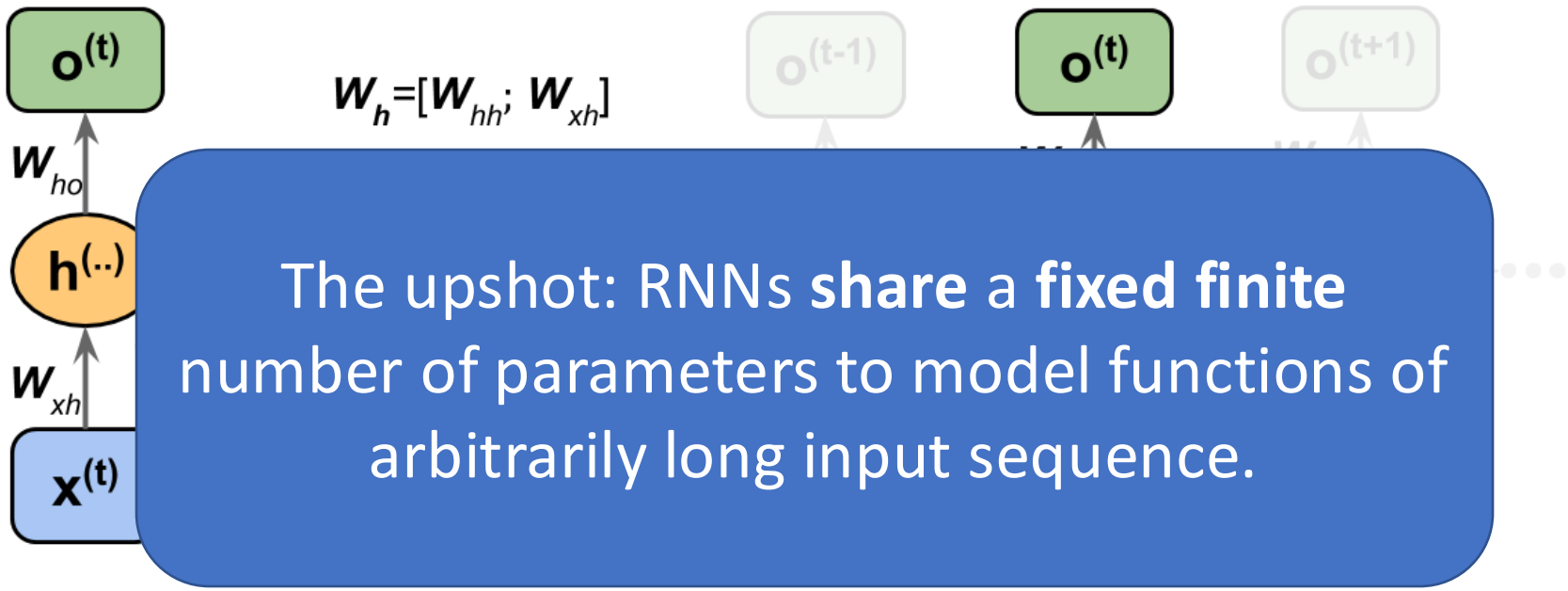


$$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{W}_{xh}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{hh}\boldsymbol{h}^{(t-1)}\right)$$

$$\boldsymbol{o}^{(t)} = \boldsymbol{W}_{ho}\boldsymbol{h}^{(t)}$$

Where σ is a nonlinear activation. Bias terms omitted.

Unfolding a Recurrent Neural Network



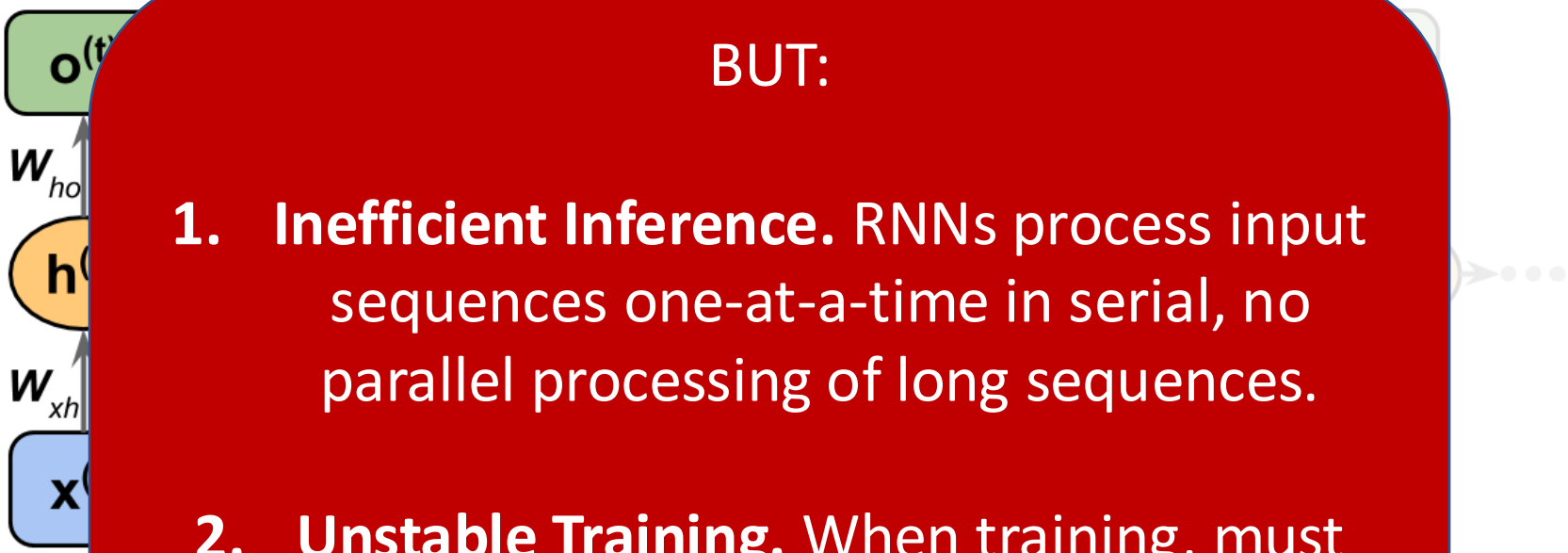
The upshot: RNNs **share** a **fixed finite** number of parameters to model functions of arbitrarily long input sequence.

$$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{W}_{xh}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{hh}\boldsymbol{h}^{(t-1)}\right)$$

$$\boldsymbol{o}^{(t)} = \boldsymbol{W}_{ho}\boldsymbol{h}^{(t)}$$

Where σ is a nonlinear activation. Bias terms omitted.

Unfolding a Recurrent Neural Network



BUT:

1. **Inefficient Inference.** RNNs process input sequences one-at-a-time in serial, no parallel processing of long sequences.

2. **Unstable Training.** When training, must backpropagate through computations proportional to sequence length – vanishing or exploding grading problems.

$\boldsymbol{h}^{(t)}$

$\boldsymbol{o}^{(t)} =$ omitted.

Autoregressive Sequence Generation: Where We're Going

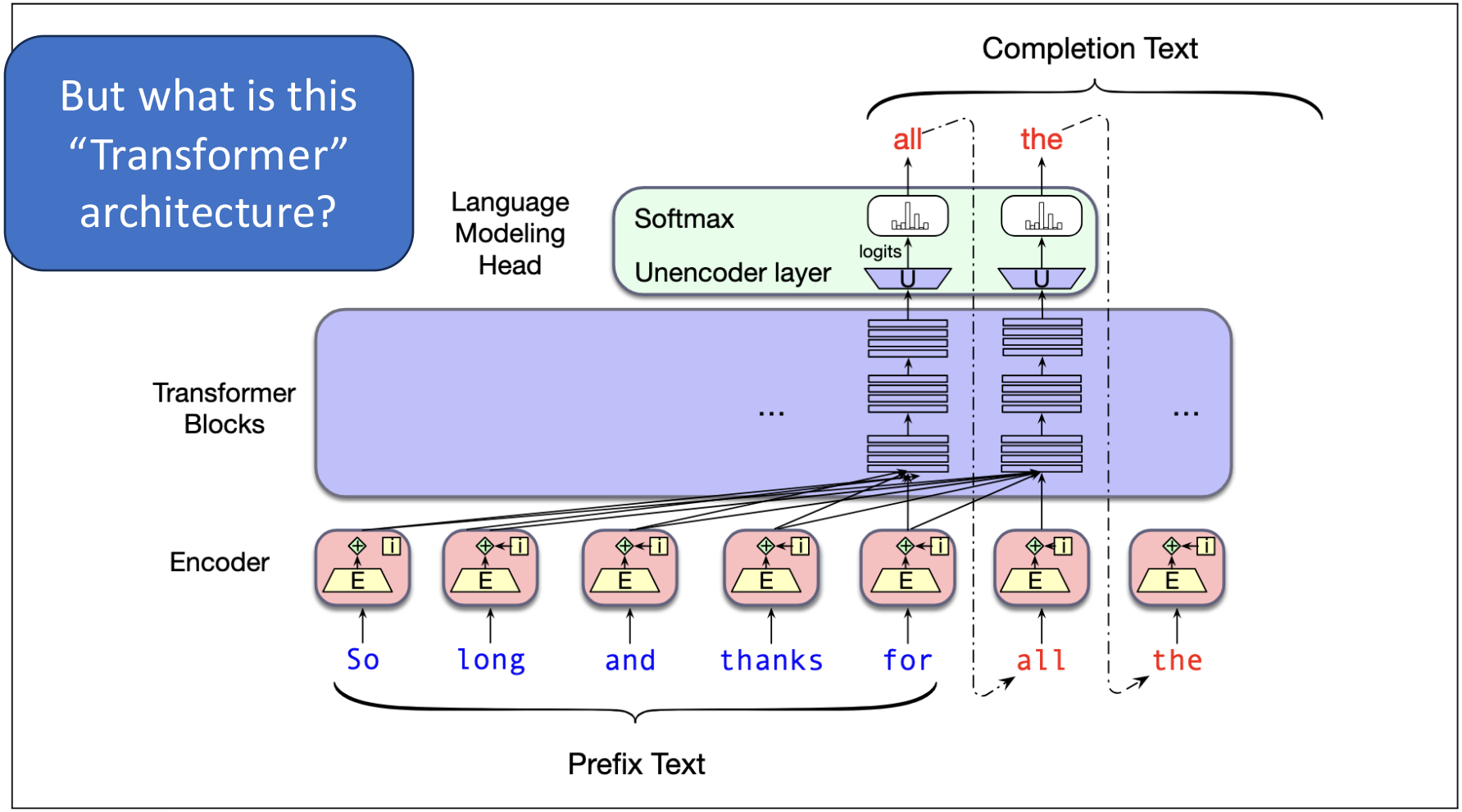


Figure 10.1 Left-to-right (also called autoregressive) text completion with transformer-based large language models. As each token is generated, it gets added onto the context as a prefix for generating the next token.

Attention is All You Need

Attention is All you Need

Part of Advances in Neural Information Processing Systems 30 (NIPS 2017)

Bibtex Metadata Paper Reviews

Over 136,000 citations in 7 years!

Authors

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

Abstract

The dominant sequence transduction models are based on complex recurrent orconvolutional neural networks in an encoder and decoder configurat o connect the encoder and decoder through an attentionm echanisms. ecture based solely onan attention mec n recurrence and convolutions entirely Experiments on two machine translation tasks sho uperiorin quality while being ly less timeto train. Our single model with 165 million parameters, ach slation, improving over the existing best ensemble result by over 1 BLEU. On English-to-French translation, we outperform the l by 0.7 BLEU, achieving a BLEU score of 41.1.

"simpler" – no recurrence

Better NLP performance

More parallelizable (GPU) training and inference

Originally used for machine translation

Transformer
The Big Picture

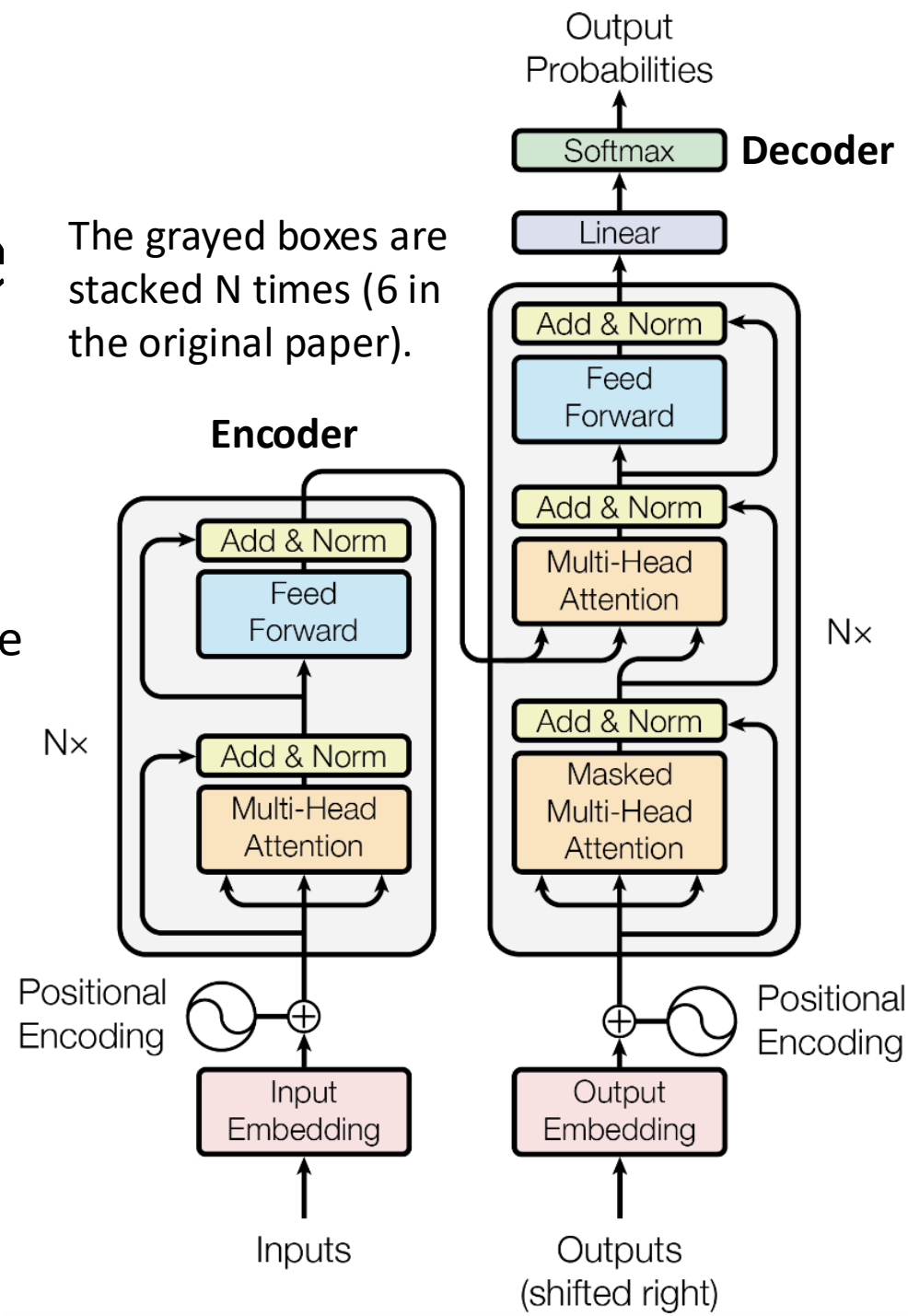
We already know some things…

- Feed Forward blocks: MLP with a single hidden layer.
- Elements of the sequence are *embedded* as a numerical vector.

We need to understand…

1. Multi-Head Attention
2. Add & Norm
3. Positional Encoding

The grayed boxes are stacked N times (6 in the original paper).



Input: Sequence of Tokens as a Matrix of Embedding Vectors

it

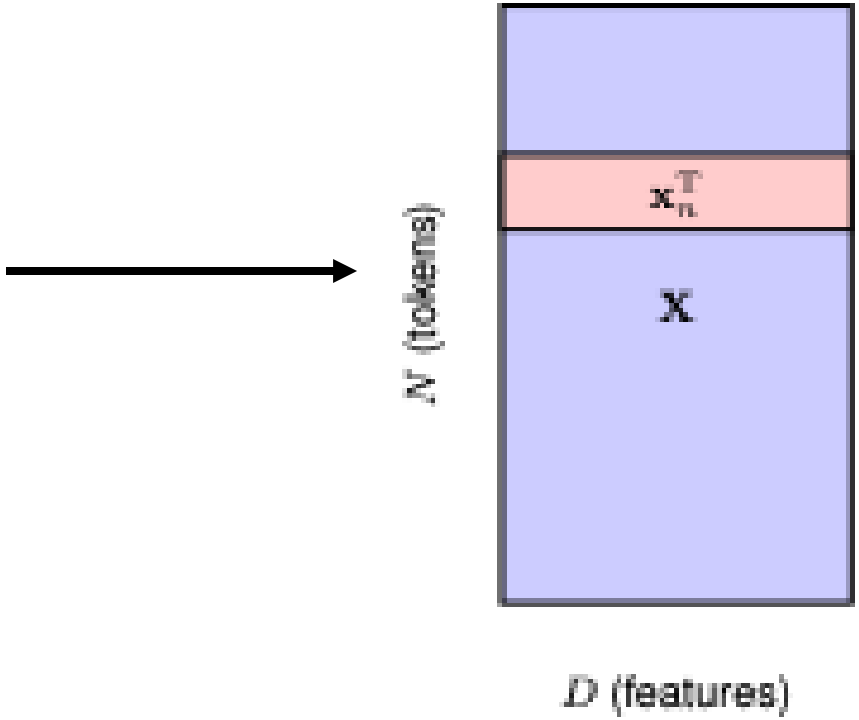
was

the

best

of

times



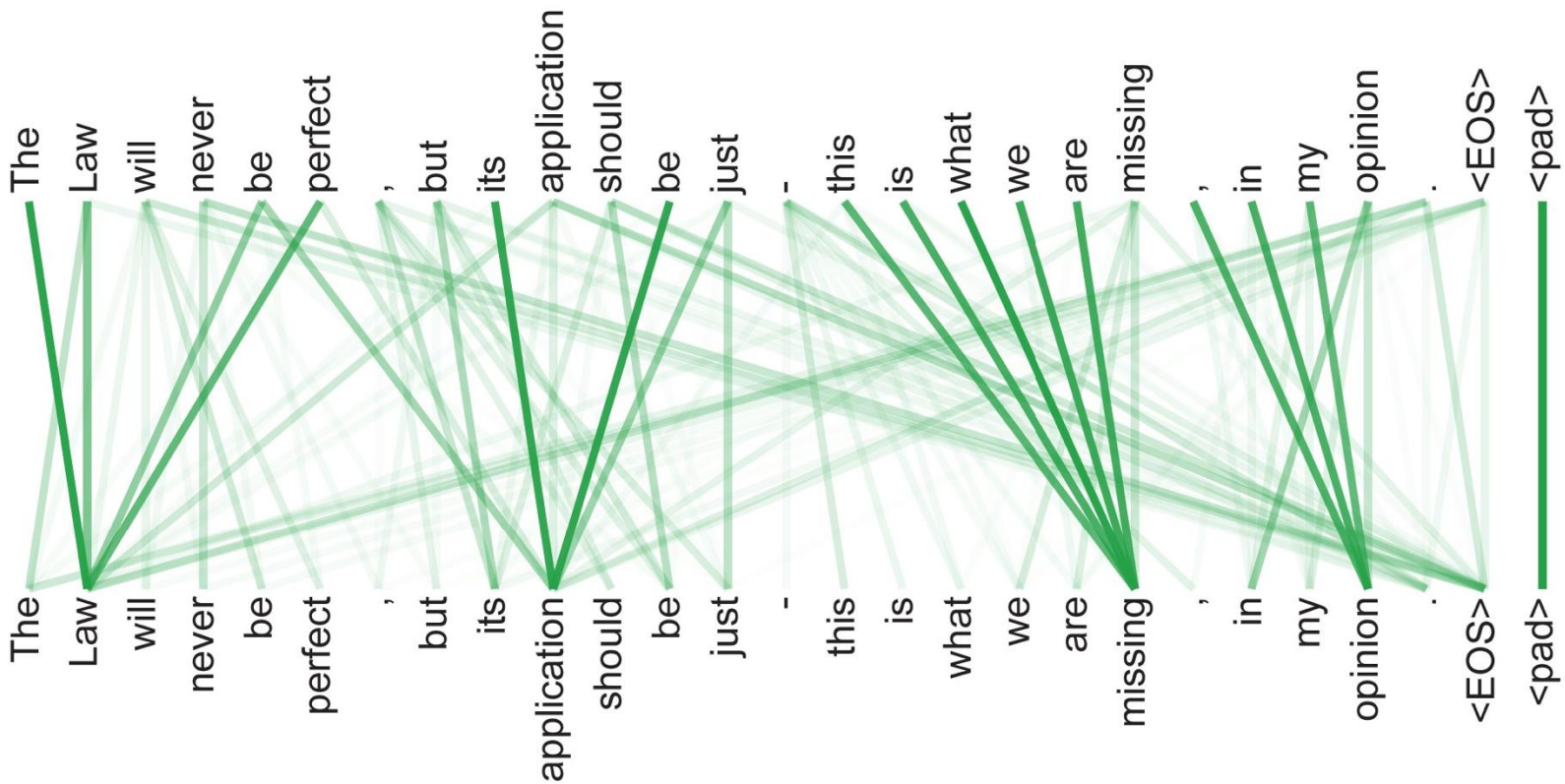
a.k.a. embedding dimension

Every transformer block outputs a transformed matrix of the same shape.

A *batch* of sequences will then become a rank 3 tensor.
But let's focus on a single input sequence for simplicity.

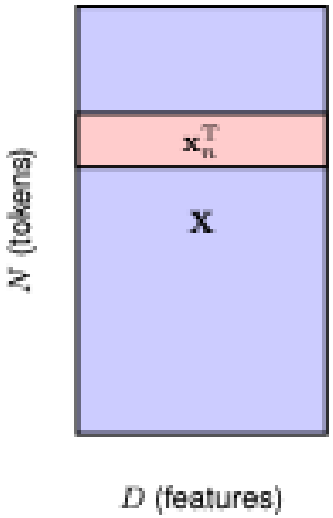
The Problem with Static Context

What context is relevant ***depends*** on what you're trying to predict.



More generally, each element of a sequence might be best contextualized by *different* combinations of other elements.

Self-Attention



$$\boldsymbol{y}_n = \sum_{i=1}^{N} \alpha_{in} \boldsymbol{x}_i$$

that is, each output vector is a weighted average of the input vectors.

where $\alpha_{in} = \dfrac{e^{\boldsymbol{x}_i^T \boldsymbol{U} \boldsymbol{x}_n}}{\sum_{i=1}^{N} e^{\boldsymbol{x}_i^T \boldsymbol{U} \boldsymbol{x}_n}}$

With $\boldsymbol{U}$ being a matrix of learnable weights.

And the α coefficients are the attention weights

To expand the model capacity of the attention mechanism, we now use 3 different learnable weight matrices:

- The query weights: $\mathbf{W}^{(q)}$
- The key weights: $\mathbf{W}^{(k)}$
- The value weights: $\mathbf{W}^{(v)}$

Common to choose all to be D by D matrices, the embedding dimension.

Self-Attention Parameters

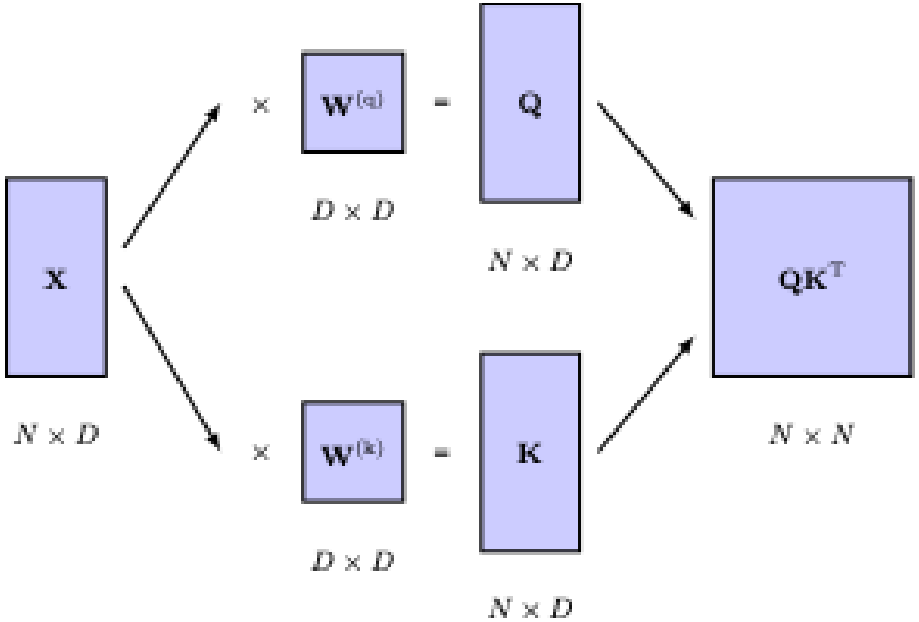
To expand the model capacity of the attention mechanism, we now use 3 different learnable weight matrices:

- The query weights: $\mathbf{W}^{(q)}$
- The key weights: $\mathbf{W}^{(k)}$
- The value weights: $\mathbf{W}^{(v)}$

Common to choose all to be D by D matrices, the embedding dimension.

Given weights and input matrix $\mathbf{X}$ we can form:

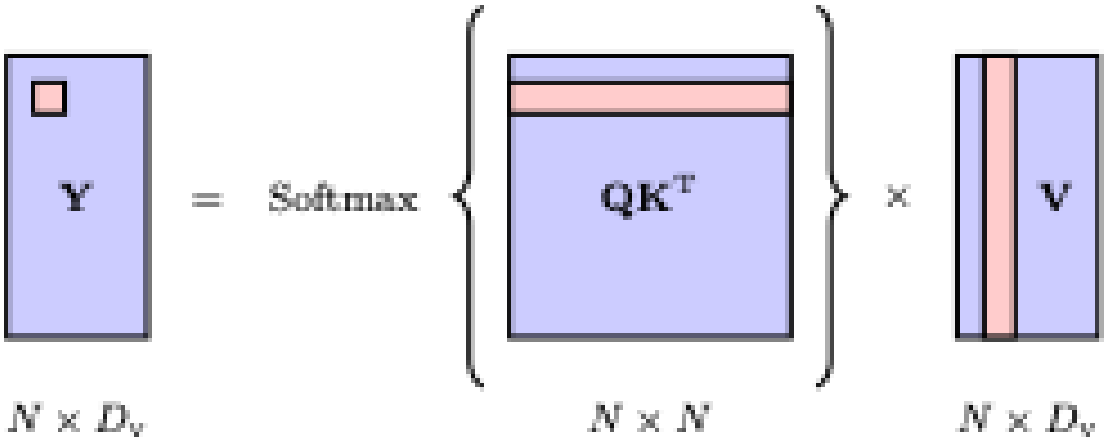
- Query matrix $\mathbf{Q} \coloneqq \mathbf{X}\mathbf{W}^{(q)}$
- Key matrix $\mathbf{K} \coloneqq \mathbf{X}\mathbf{W}^{(k)}$
- Value matrix $\mathbf{V} \coloneqq \mathbf{X}\mathbf{W}^{(v)}$



Self-Attention Outputs

$$\mathbf{Y} = \mathrm{Softmax}\left(\mathbf{Q}\mathbf{K}^{\mathrm{T}}\right)\mathbf{V}$$

Where Softmax is applied to normalize each row to be nonnegative values summing to 1.

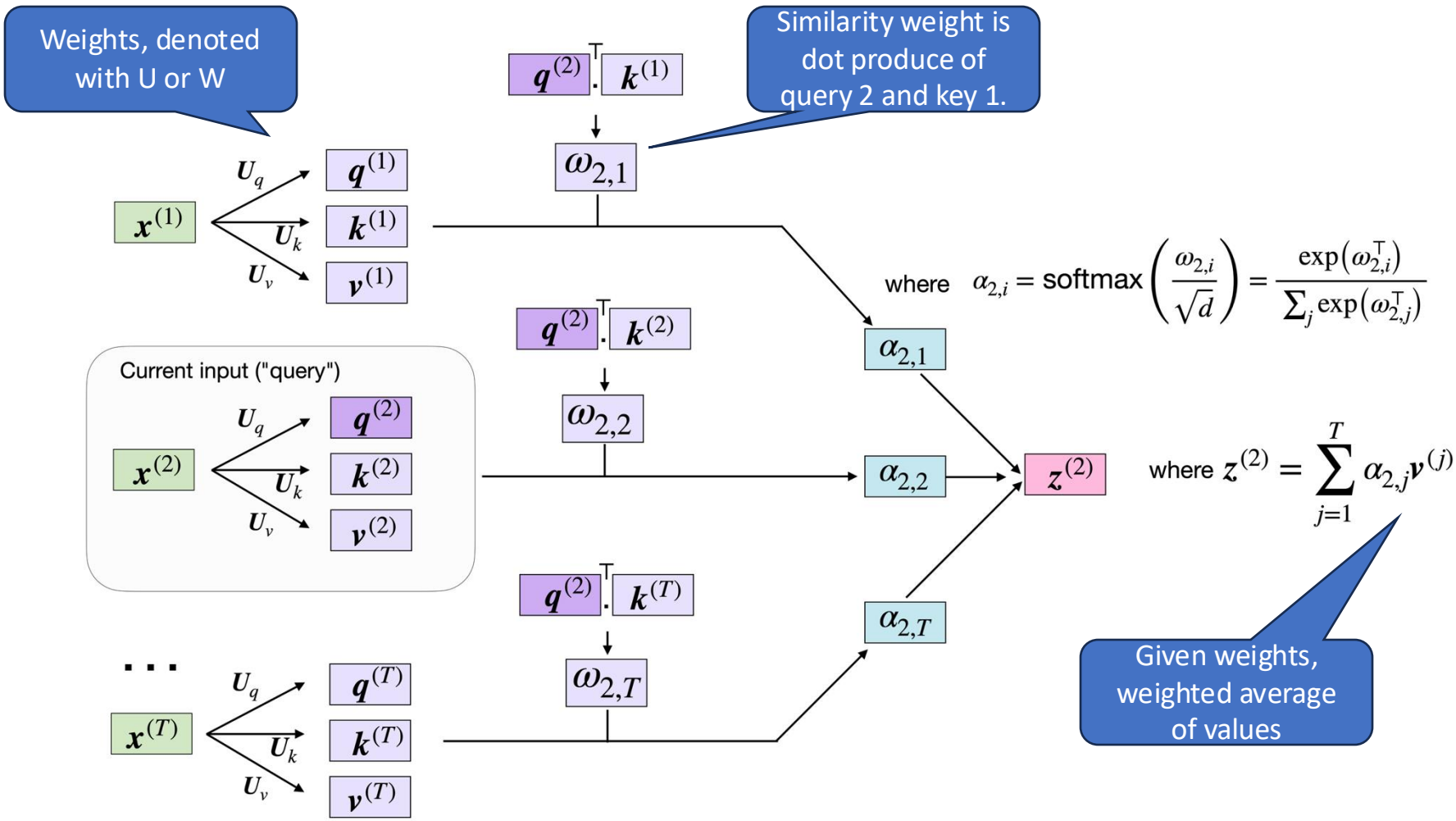


Bias terms may be included but are not shown for simplicity.

$D_v = D$ is a common choice

Note that **Q**, **K**, and **V** are intermediate results, and not learnable weights.

Zooming In: Interpreting Query, Key, Value Matrices



Scaled Self-Attention

Algorithm 12.1: Scaled dot-product self-attention

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$

Weight matrices $\{\mathbf{W}^{(\mathrm{q})}, \mathbf{W}^{(\mathrm{k})}\} \in \mathbb{R}^{D \times D_\mathrm{k}}$ and $\mathbf{W}^{(\mathrm{v})} \in \mathbb{R}^{D \times D_\mathrm{v}}$

Output: $\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \in \mathbb{R}^{N \times D_\mathrm{v}} : \{\mathbf{y}_1, \ldots, \mathbf{y}_N\}$

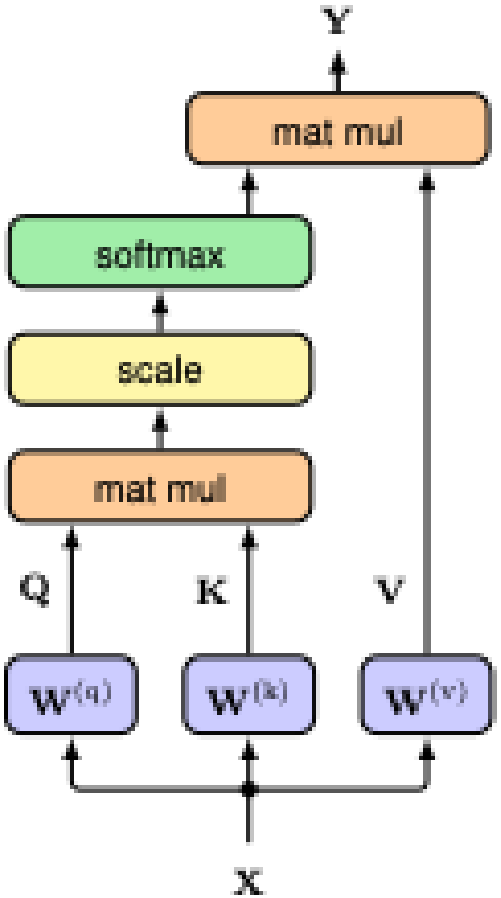
$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(\mathrm{q})}$ `// compute queries` $\mathbf{Q} \in \mathbb{R}^{N \times D_\mathrm{k}}$

$\mathbf{K} = \mathbf{X}\mathbf{W}^{(\mathrm{k})}$ `// compute keys` $\mathbf{K} \in \mathbb{R}^{N \times D_\mathrm{k}}$

$\mathbf{V} = \mathbf{X}\mathbf{W}^{(\mathrm{v})}$ `// compute values` $\mathbf{V} \in \mathbb{R}^{N \times D}$

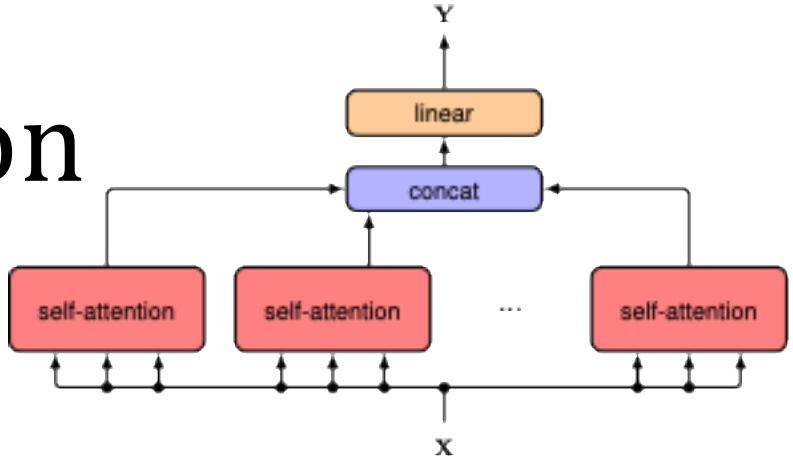
return $\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{Softmax}\left[\frac{\mathbf{Q}\mathbf{K}^\mathrm{T}}{\sqrt{D_\mathrm{k}}}\right]\mathbf{V}$

$D_k = D$ is a common choice



Scaling by $\sqrt{D_k}$ tends to keep the gradient magnitude of the softmax from becoming too small during training.

Multi-head attention



Concatenate H scaled attention mechanisms, also called *heads*. Output projection back to embedding dimension.

Algorithm 12.2: Multi-head attention

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$
Query weight matrices $\{\mathbf{W}_1^{(\mathrm{q})}, \ldots, \mathbf{W}_H^{(\mathrm{q})}\} \in \mathbb{R}^{D \times D}$
Key weight matrices $\{\mathbf{W}_1^{(\mathrm{k})}, \ldots, \mathbf{W}_H^{(\mathrm{k})}\} \in \mathbb{R}^{D \times D}$
Value weight matrices $\{\mathbf{W}_1^{(\mathrm{v})}, \ldots, \mathbf{W}_H^{(\mathrm{v})}\} \in \mathbb{R}^{D \times D_{\mathrm{v}}}$
Output weight matrix $\mathbf{W}^{(\mathrm{o})} \in \mathbb{R}^{H D_{\mathrm{v}} \times D}$
Output: $\mathbf{Y} \in \mathbb{R}^{N \times D} : \{\mathbf{y}_1, \ldots, \mathbf{x}_N\}$

```
// compute self-attention for each head (Algorithm 12.1)
```

for $h = 1, \ldots, H$ **do**
  $\mathbf{Q}_h = \mathbf{X}\mathbf{W}_h^{(\mathrm{q})}, \quad \mathbf{K}_h = \mathbf{X}\mathbf{W}_h^{(\mathrm{k})}, \quad \mathbf{V}_h = \mathbf{X}\mathbf{W}_h^{(\mathrm{v})}$
  $\mathbf{H}_h = \mathrm{Attention}\left(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h\right)$ // $\mathbf{H}_h \in \mathbb{R}^{N \times D_{\mathrm{v}}}$
end for
$\mathbf{H} = \mathrm{Concat}\left[\mathbf{H_1}, \ldots, \mathbf{H_N}\right]$ // concatenate heads
return $\mathbf{Y}(\mathbf{X}) = \mathbf{H}\mathbf{W}^{(\mathrm{o})}$

Common to choose $D_v = D/H$

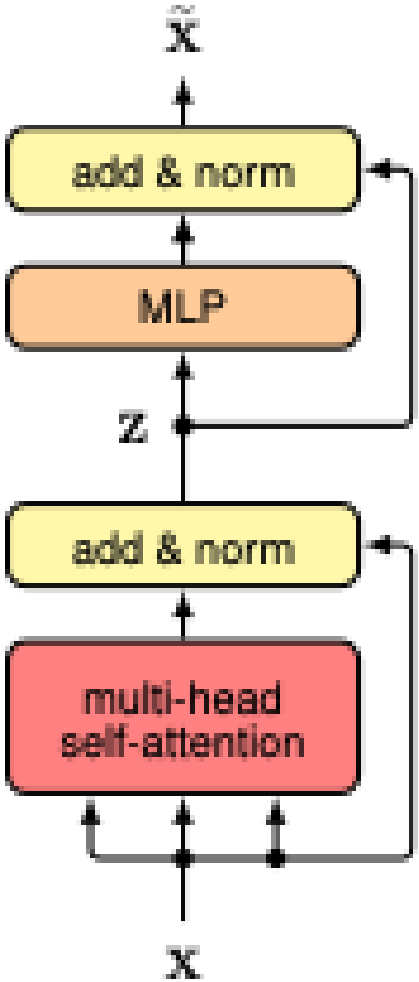
Increases model capacity: conceptually like multiple filters in a convolutional network

This is the core of the transformer architecture.

Add & Norm: Residual or "Skip" Connection

$$\mathbf{Z} = \mathrm{Norm}(\mathrm{Attention}(\mathbf{X}) + \mathbf{X})$$

The **residual** add (element-wise)



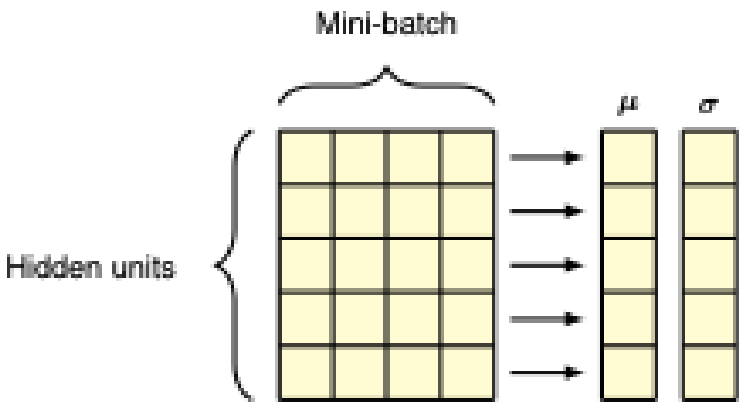
Used in very deep architectures to mitigate noisy exploding/vanishing gradient.

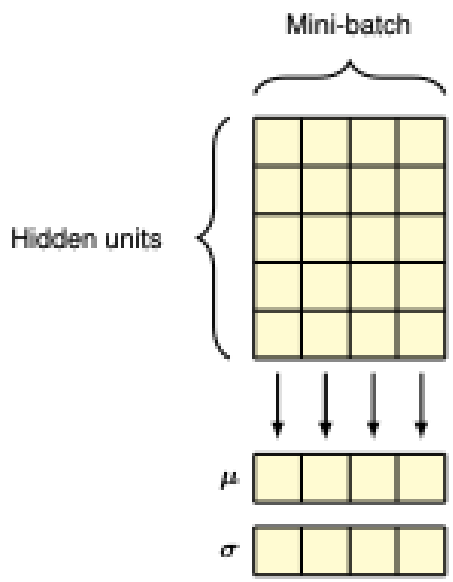
Stabilizes learning.

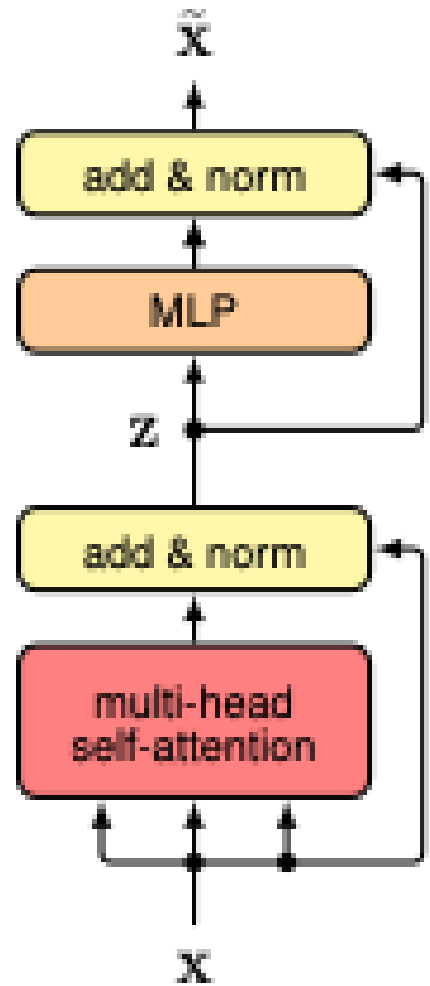
Add & Norm: Batch and Layer Normalization

$$\mathbf{Z} = \text{Norm}(\text{Attention}(\mathbf{X}) + \mathbf{X})$$

Typically layer normalization in a Transformer







Compute mean μ and standard deviation σ across mini-batch (batch norm) or hidden units (layer norm).

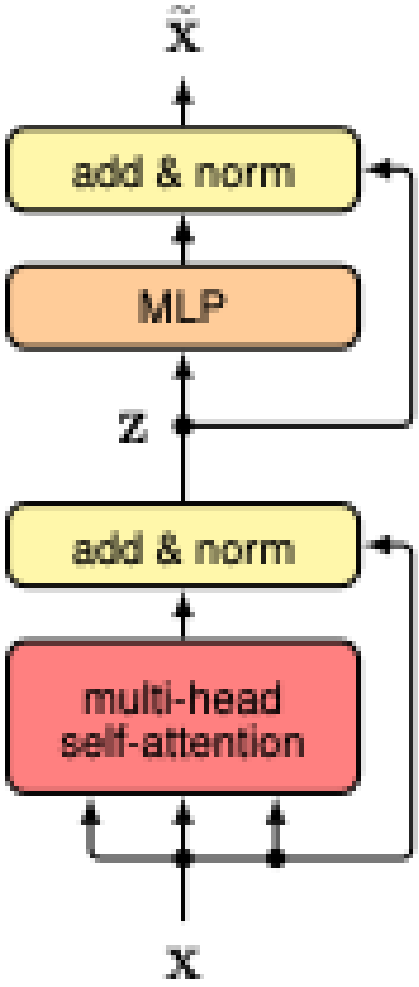
Subtract μ and divide by σ for each hidden unit (batch norm) or for each example (layer norm)

Standard Transformer Block

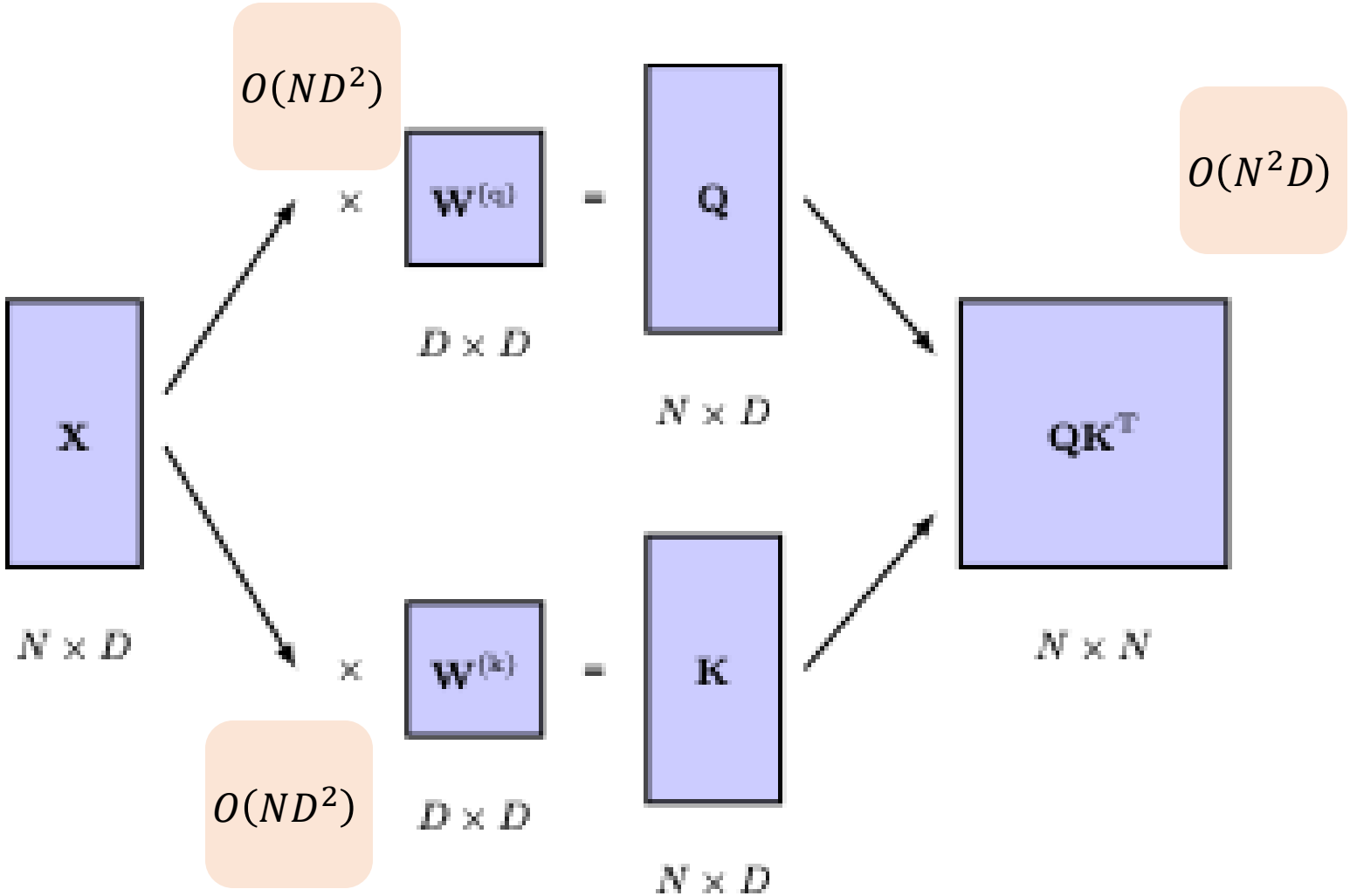
Returns a matrix of the same shape as the input.

1. Multi-head self-attention
2. Residual add, normalize
3. MLP with a single hidden layer
4. Another residual add and normalize

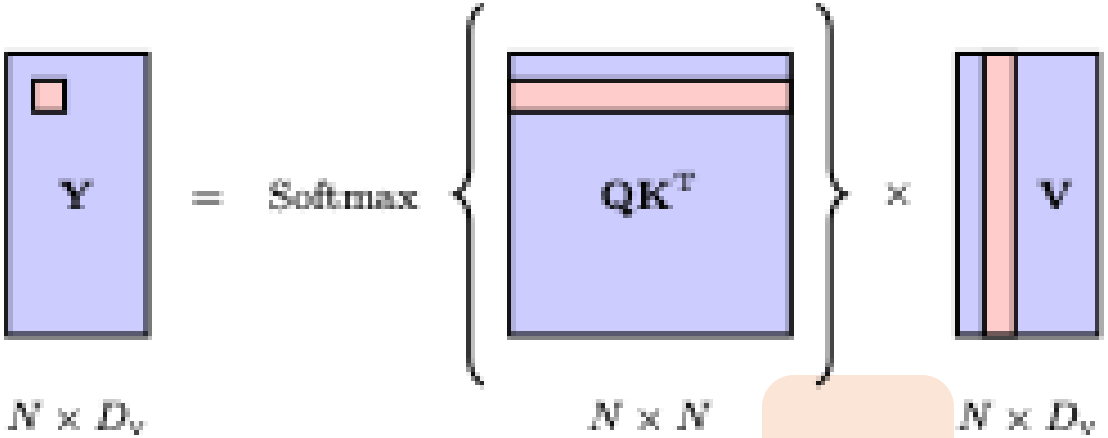
Stack these blocks together



Computational Complexity

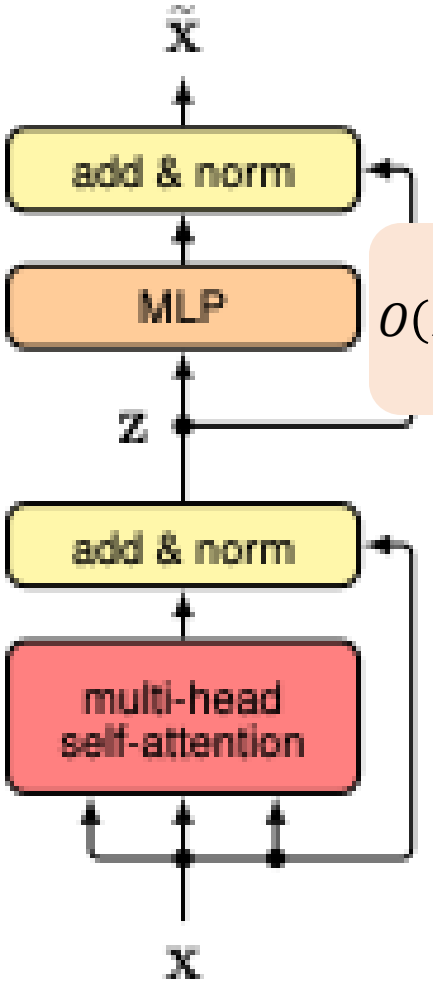


Computational Complexity



$O(N^2D)$

$D_v = D$ is a common choice



$O(ND^2)$

Overall, a transformer block has computational complexity $O(N^2D + ND^2)$.

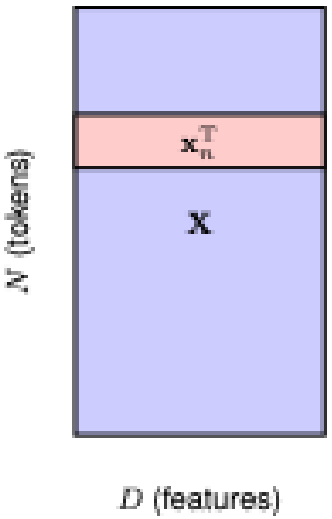
Every runtime dominating operation is a matrix multiplication: Maximally parallelizable.

Order Matters

Consider the inputs:

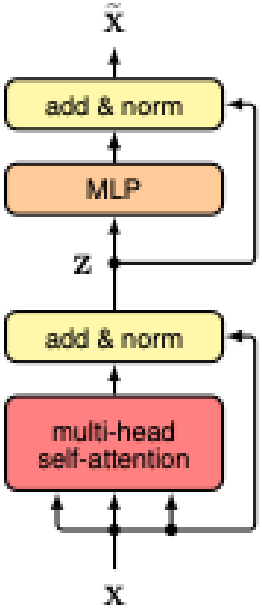
a) The food was good, not bad at all

b) The food was bad, not good at all



But a transformer block would give the **same** transformation just with the rows permuted.

We explicitly add a ***positional encoding/embedding***. Without this a transformer could not distinguish



Positional Encoding/Embedding Example

A single linear layer embeds each position index into a vector of length D (the embedding dimension). Add this to the word embedding for the token at that position, also of length D.

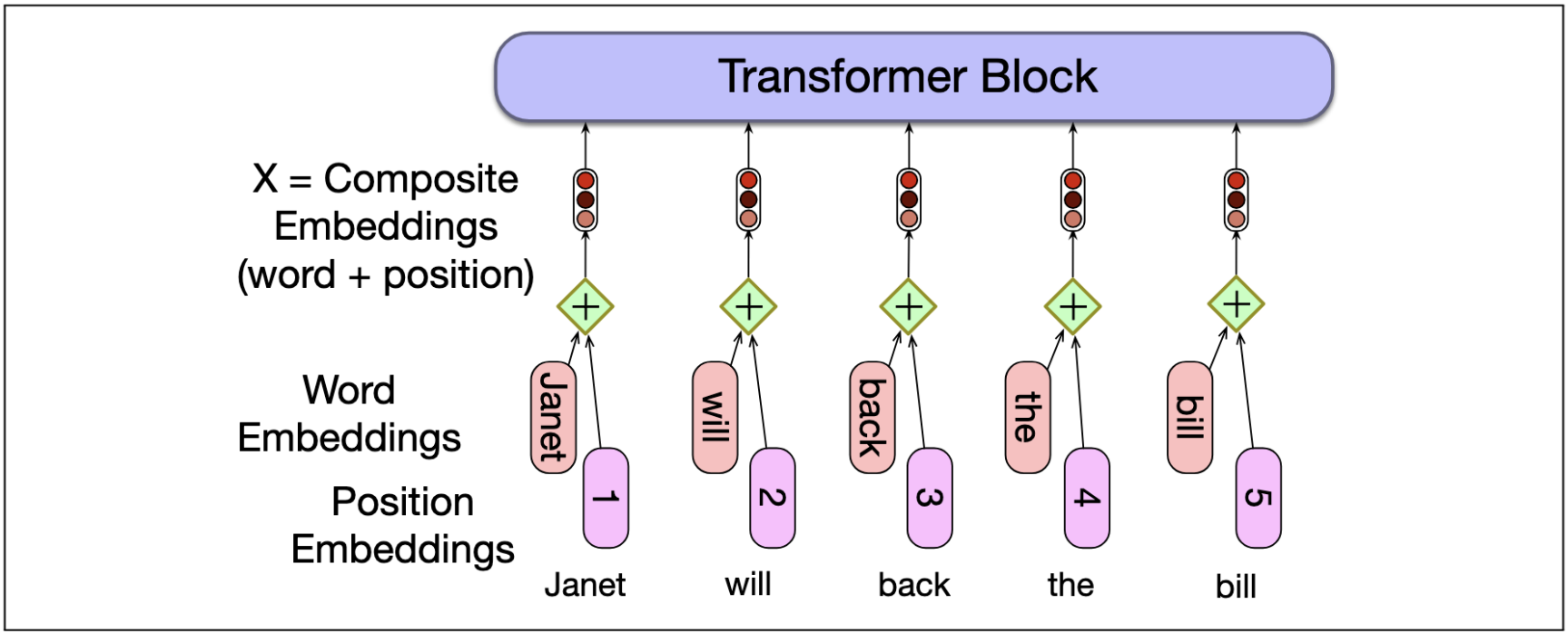


Figure 9.13 A simple way to model position: add an embedding of the absolute position to the token embedding to produce a new embedding of the same dimensionality.

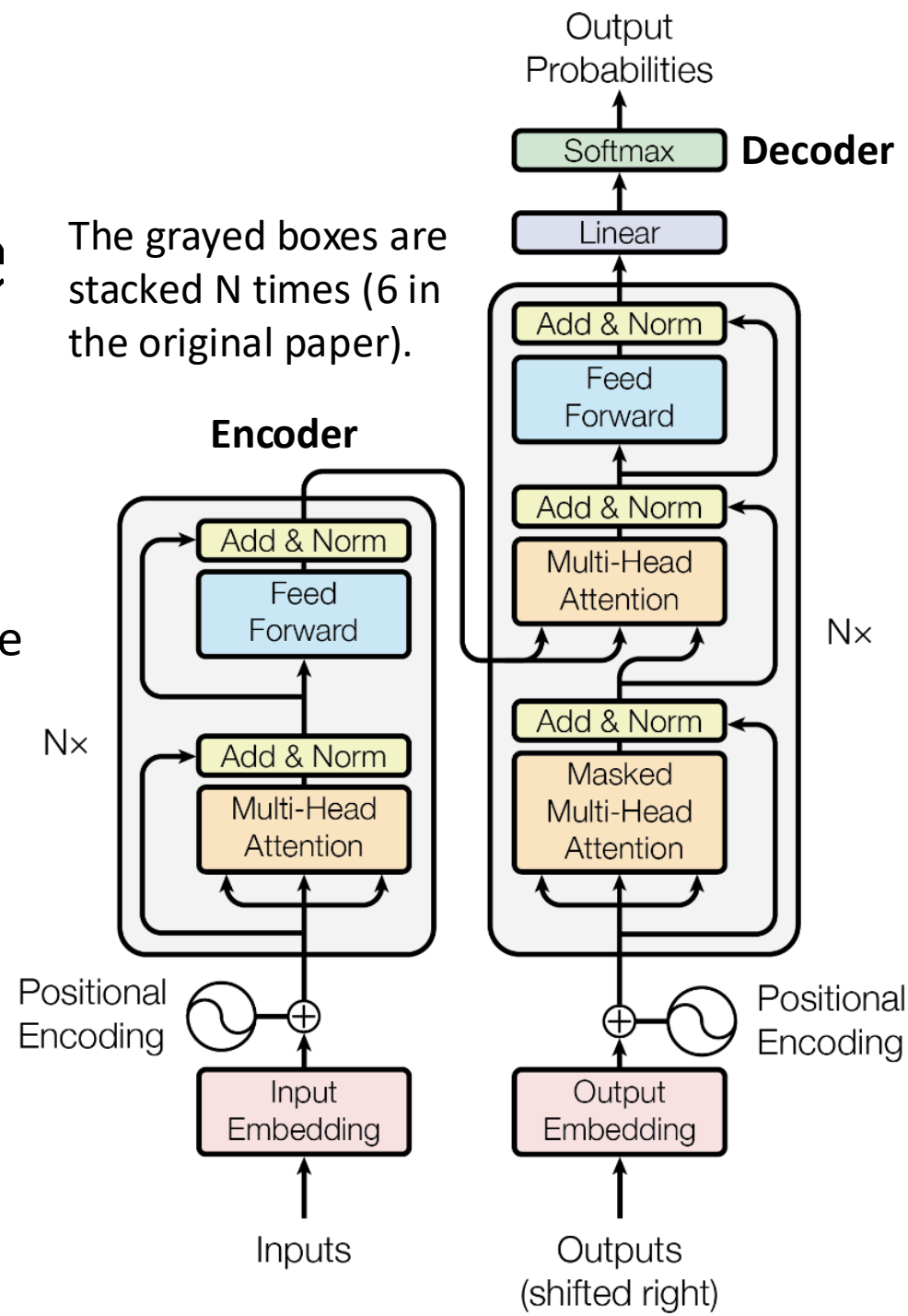
Transformer
The Big Picture

We already know some things…

- Feed Forward blocks: MLP with a single hidden layer.
- Elements of the sequence are *embedded* as a numerical vector.

We now understand

1. Multi-Head Attention
2. Add & Norm
3. Positional Encoding

Next time: How to use it!

The grayed boxes are stacked N times (6 in the original paper).